# LAB # 11: YOLO based Image Segmentation

## Objective

The objective of this lab is to understand and implement YOLO-based instance segmentation using a pre-trained YOLOv8 model. The lab focuses on enabling students to perform object segmentation, analyze model performance using standard evaluation metrics, and apply segmentation techniques to real-time video streams.

## Introduction

Image segmentation is a fundamental task in computer vision that involves assigning a class label to each pixel in an image. Unlike object detection, which only provides bounding boxes around objects, segmentation provides a more detailed understanding by identifying the exact shape and boundaries of objects.

YOLO (You Only Look Once) is a state-of-the-art, real-time object detection algorithm that has been extended to perform instance segmentation. In YOLOv8, segmentation is achieved by adding a specialized segmentation head that predicts pixel-level masks along with bounding boxes and class labels in a single forward pass. This makes YOLO highly efficient and suitable for real-time applications such as autonomous driving, surveillance, and medical imaging.

## Theoretical Background

The YOLOv8 segmentation model is composed of three major components: the Backbone, Neck, and Head, each playing a critical role in the overall architecture.

The Backbone is responsible for feature extraction. It processes the input image through multiple convolutional layers to capture low-level and high-level features such as edges, textures, and object

patterns. These extracted features are represented as feature maps, which are then passed to the next stage.

The Neck performs feature aggregation. It combines feature maps from different scales to ensure that both small and large objects are effectively detected. This multi-scale feature fusion enhances the model's ability to generalize across varying object sizes and improves segmentation accuracy.

The Head is the final component where predictions are made. In YOLOv8 segmentation, the head outputs bounding boxes, class probabilities, and segmentation masks. The segmentation process is based on the concept of prototype masks and mask coefficients. Prototype masks represent general shapes, while coefficients determine how these shapes are combined to form the final object mask. This approach allows YOLO to generate accurate segmentation masks efficiently without heavy computational overhead.

**Methodology**

The implementation of YOLO-based segmentation involves several steps, starting from setting up the environment to executing inference and evaluating performance.

Initially, the required libraries are installed and imported into the Python environment. The pre-trained YOLOv8 segmentation model is then loaded. This model has been trained on large-scale datasets such as COCO and is capable of segmenting multiple object classes.

Once the model is loaded, segmentation is performed on a sample image. The model processes the image and outputs bounding boxes, class labels, and corresponding segmentation masks. These masks highlight the exact regions occupied by objects in the image.

After performing inference, the next step is to evaluate the model's performance. Standard evaluation metrics such as mean Average Precision (mAP), precision, and recall are used to assess how well the model performs in detecting and segmenting objects.

Finally, the model is applied to real-time data using a webcam or video stream. This demonstrates the real-time capability of YOLO segmentation, where objects are detected and segmented instantly as the video is processed frame by frame.

**Implementation**

**Step 1: Install Required Libraries**

```
pip install ultralytics
```

**Step 2: Import Libraries and Load Model**

```
from ultralytics import YOLO


model = YOLO("yolov8n-seg.pt")
```

**Step 3: Perform Segmentation on Image**

```
results = model("image.jpg", show=True)
```

This step performs inference on the input image and displays the segmented output, including masks and bounding boxes.

**Step 4: Evaluate Model Performance**

```
metrics = model.val()

print(metrics)
```

This step computes evaluation metrics such as mAP, precision, and recall.

**Step 5: Apply Real-Time Segmentation**

```
model.predict(source=0, show=True)
```

This enables segmentation using a webcam feed. For video input, a file path can be provided instead of 0.

**Lab Tasks**

1- Load a pre-trained YOLOv8 segmentation model and perform segmentation on a sample image. Display the output with segmentation masks.
2- Evaluate the segmentation model using standard metrics such as mAP, precision, and recall. Analyze the results obtained.
3- Apply YOLO based segmentation on a real time video stream or webcam feed. Observe how the model performs in real-time conditions.
4- Train a YOLOv8 segmentation model on a custom dataset (e.g., pothole or road damage dataset). Prepare the dataset, configure the YAML file, and train the model for specified number of epochs. Evaluate the trained model on unseen images and display results with segmentation masks, along with training curves and performance metrics.